

UNIT-5

OBJECT ORIENTED PROGRAMMING AND EXCEPTION HANDLING.

Python has been an object-oriented language since it existed. Because of this, creating and using classes and objects are downright easy.

Overview of OOP Terminology

- **Class:** A user-defined prototype for an object that defines a set of attributes that characterize any object of the class. The attributes are data members (class variables and instance variables) and methods, accessed via dot notation.
- **Class variable:** A variable that is shared by all instances of a class. Class variables are defined within a class but outside any of the class's methods. Class variables are not used as frequently as instance variables are.
- **Data member:** A class variable or instance variable that holds data associated with a class and its objects.
- **Function overloading:** The assignment of more than one behavior to a particular function. The operation performed varies by the types of objects or arguments involved.
- **Instance variable:** A variable that is defined inside a method and belongs only to the current instance of a class.
- **Inheritance:** The transfer of the characteristics of a class to other classes that are derived from it.
- **Instance:** An individual object of a certain class. An object obj that belongs to a class Circle, for example, is an instance of the class Circle.
- **Instantiation:** The creation of an instance of a class.
- **Method:** A special kind of function that is defined in a class definition.
- **Object:** A unique instance of a data structure that's defined by its class. An object comprises both data members (class variables and instance variables) and methods.
- **Operator overloading:** The assignment of more than one function to a particular operator.

Creation of Class:

A class is created with the keyword **class** and then writing the class name. The simplest form of class definition looks like this:

```
class Class Name:
    <statement-1>
    .
    .
    <statement-N>
```

Class definitions, like function definitions (def statements) must be executed before they have any effect.

Example:

<pre>class Student: def __init__(self): self.name="hari" self.branch="ccn" def display(self): print (self.name) print(self.branch) obj=Student() obj.__init__() obj.display()</pre>	Output: hari ccn
---	-------------------------------

Creating an Object:

- To create an instance, the following **syntax** is used:

instance name = Class name()

- So, to create an instance to the Student class, we can write as:

s1 = Student ()

- Here “s1” represents the instance name. When we create an instance like this, the following steps will take place internally:

1. First of all, a block of memory is allocated on heap. How much memory is to be allocated is decided from the attributes and methods available in the Student class.
2. After allocating the memory block, the special method by the name “**__init__(self)**” is called internally. This method stores the initial data into the variables. Since this method is useful to construct the instance, it is called “**constructor**”.

Example :

```
class student:
    pin=123
    name="abc"
    branch="ccn"
    def read(self):
        print("student is reading")
    def write(self):
        print("student is writing")
obj1=student()
print("name=",obj1.name)
print("pin=",obj1.pin)
obj1.read()
obj1.write()
```

output:

```
name= abc
pin=123
student is reading
student is writing
```

Self variable:

“**Self**” is a default variable that contains the memory address of the instance of the current class. When an instance to the class is created, the instance name contains the memory location of the instance. This memory location is internally passed to “self”.

For example, we create an instance to student class as:

s1 = Student()

Here, “s1” contains the memory address of the instance. This memory address is internally and by default passed to “self” variable. Since “self” knows the memory address of the instance, it can refer to all the members of the instance.

We use “self” in two easy:

- The self variable is used as first parameter in the constructor as:

def __init__(self):

In this case, “self” can be used to refer to the instance variables inside the constructor.

- “self” can be used as first parameter in the instance methods as:

def display(self):

Here, display() is instance method as it acts on the instance variables. If this method wants to act on the instance variables, it should know the memory location of the instance variables. That memory location is by default available to the display() method through “**self**”.

Constructor:

A constructor is a special method that is used to initialize the instance variables of a class. In the constructor, we create the instance variables and initialize them with some starting values. The first parameter of the constructor will be “self” variable that contains the memory address of the instance.

```
def __init__(self):  
    self.name = "hari"  
    self.branch = "ccn"
```

Here, the constructor has only one parameter, i.e. “self” using “**self.name**” and “**self.branch**”, we can access the instance variables of the class. A constructor is called at the time of creating an instance. So, the above constructor will be called when we create an instance as:

```
s1 = Student()
```

Let’s take another example, we can write a constructor with some parameters in addition to “self” as:

```
def __init__(self, n = "", b = ""):  
    self.name = n  
    self.branch = b
```

Here, the formal arguments are “n” and “b” whose default values are given as “” (None) and “” (None). Hence, if we do not pass any values to constructor at the time of creating an instance, the default values of those formal arguments are stored into name and branch variables. For example,

```
s1 = Student()
```

Since we are not passing any values to the instance, None and None are stored into name and branch. Suppose, we can create an instance as:

```
s1 = Student("mothi", "CCN")
```

In this case, we are passing two actual arguments: “mothi” and “CCN” to the Student instance.

Example 1:

```
class Student:  
    def __init__(self, n="", b=""):  
        self.name = n  
        self.branch = b  
    def display(self):  
        print("Hi", self.name)  
        print("Branch", self.branch)  
s1 = Student()  
s1.display()  
s2 = Student("geethika", "CCN")  
s2.display()
```

Output:

```
Hi  
Branch  
Hi geethika  
Branch CCN
```

Example 2:

```
program1:
class method:
    def __init__(self,r,n):
        self.name=n
        self.rollno=r
    def student(self):
        print("{}={}".format(self.name,self.rollno))
obj=method("mounika",52)
obj.student()
output:
mounika=52
```

Types of Variables:

- a) The variables which are written inside a class are of 2 types: Instance Variables
- b) Class Variables or Static Variables

a) Instance Variables

Instance variables are the variables whose separate copy is created in every instance. For example, if “x” is an instance variable and if we create 3 instances, there will be 3 copies of “x” in these 3 instances. When we modify the copy of “x” in any instance, it will not modify the other two copies.

Example: A Python Program to understand instance variables.

```
class Sample:
    def __init__(self):
        self.x =10
    def modify(self):
        self.x = self.x + 1
s1=Sample()
s2=Sample()

print("x ins1=",s1.x)
print("x in s2=",s2.x)
print("x in s1=",s1.x)
print("x in s2=",s2.x)
```

Output:

```
x in s1= 10
x in s2= 10
x in s1= 11
x in s2= 10
```

Instance variables are defined and initialized using a constructor with “**self**” parameter. Also, to access instance variables, we need instance methods with “self” as first parameter. It is possible that the instance methods may have other parameters in addition to the “self” parameter. To access the instance variables, we can use **self. variable** as shown in program. It is also possible to access the instance variables from outside the class, as:

instance name. variable, e.g. **s1.x**

b) Class Variables or Static Variables

Class variables are the variables whose single copy is available to all the instances of the class. If we modify the copy of class variable in an instance, it will modify all the copies in the other instances. For example, if “x” is a class variable and if we create 3 instances, the same copy of “x” is passed to these 3 instances. When we modify the copy of “x” in any instance using a class method, the modified copy is sent to the other two instances.

Example: A Python program to understand class variables or static variables.

```
class Sample:
    x=10
    @classmethod
    def modify(cls):
        cls.x = cls.x + 1
s1=Sample()
s2=Sample()
print("x ins1=",s1.x)
print("x in s2=",s2.x)
print("s1=", s1.modify())
print("x ins1=",s1.x)
print("x in s2=",s2.x)
```

Output:

```
x in s1= 10
x in s2= 10
x in s1= 11
x in s2= 11
```

Types of methods:

We can classify the methods in the following 3 types:

- a) Instance methods
 - Accessor methods
 - Mutator methods
- b) Class methods
- c) Static methods

a) Instance Methods:

Instance methods are the methods which act upon the instance variables of the class. Instance methods are bound to instances and hence called as: **instance name. method()**. Since instance variables are available in the instance, instance methods need to know the memory address of instance. This is provided through “self” variable by default as first parameter for the instance method. While calling the instance methods, we need not pass any value to the “self” variable.

Example:

```
class Student:
    def __init__(self,n="",b=""):
        self.name=n
        self.branch=b
    def display(self):
        print ("Hi",self.name)
        print("Branch",self.branch)
s1=Student()s1.display()
Print(".....")
s2=Student("mothi","CCN")
s2.display()
```

```
print (".....")
```

- Instance methods are of two types: **accessor methods and mutator methods**.
- **Accessor methods** simply access or read data of the variables. They do not modify the data in the variables. Accessor methods are generally written in the form of `get XXXX( )` and hence they are also called **getter methods**.
- **Mutator methods** are the methods which not only read the data but also modify them. They are written in the form of `set XXXX( )` and hence they are also called **setter methods**.

Example:

```
class Student:
    def set Name(self,n):
        self.name = n
    def setBranch(self,b):
        self. branch = b
    def getName(self):
        return self.name
    def getBranch(self):
        return self.branch
s=Student()
name=input("Enter Name: ")
branch=input("Enter Branch: ")
s.setName(name)
s.setBranch(branch)
print(s.getName())
print(s.getBranch())
```

output:

```
Enter Name: Lakshmi
Enter Branch: CCN
Lakshmi
CCN
```

b) Class methods:

These methods act on class level. Class methods are the methods which act on the class variables or static variables. These methods are written using **@classmethod** decorator above them. By default, the first parameter for class methods is “cls” which refers to the class itself.

For example, “cls.var” is the format to the class variable. These methods are generally called using **classname.method()**. The processing which is commonly needed by all the instances of class is handled by the class methods.

Example:

```
class Bird:
    wings = 2
    @class method
    def fly(cls,name):
        print(name,"flies with",cls.wings,"wings")
Bird.fly("parrot") #display "parrot flies with 2 wings"
Bird.fly("sparrow") #display "sparrow flies with 2 wings"
```

Output:

```
parrot flies with 2 wings
sparrow flies with 2 wings
```

c) Static methods:

We need static methods when the processing is at the class level but we need not involve the class or instances. Static methods are used when some processing is related to the class but does not need the class or its instances to perform any work.

For example, setting environmental variables, counting the number of instances of the class or changing an attribute in another class, etc. are the tasks related to a class.

Such tasks are handled by static methods. Static methods are written with decorator **@staticmethod** above them. Static methods are called in the form of **classname.method ()**.

Example :

```
class MyClass:
    n=0
    def __init__(self):
        MyClass.n = MyClass.n + 1
    def noObjects ( ):
        print("No. of instances created: ",MyClass.n)
m1=MyClass()
m2=MyClass()
m3=MyClass()
MyClass.NoObjects()
```

Output:

No. of instances created:3

Inheritance:

- Software development is a team effort. Several programmers will work as a team to develop software.
- When a programmer develops a class, he will use its features by creating an instance to it. When another programmer wants to create another class which is similar to the class already created, then he need not create the class from the scratch. He can simply use the features of the existing class in creating his own class.
- Deriving new class from the super class is called **inheritance**.
- The child class inherits the attributes of its parent class, and you can use those attributes as if they were defined in the child class.
- A child class can also override data members and methods from the parent.

Syntax:

```
class Subclass(Base Class):
    <class body>
```

- When an object is to Sub Class is created, it contains a copy of Base Class within it. This means there is a relation between the Base Class and Sub Class objects.
- By using inheritance, a programmer can develop classes very easily. Hence programmer's productivity is increased. Productivity is a term that refers to the code developed by the programmer in a given span of time.
- If the programmer used inheritance, he will be able to develop more code in less time.

The super() method:

- **super()** is a built-in method which is useful to call the super class constructor or methods from the sub class.

- Any constructor written in the super class is not available to the sub class if the sub class has a constructor.
- Then how can we initialize the super class instance variables and use them in the sub class? This is done by calling super class constructor using `super( )` method from inside the sub class constructor.
- `super( )` is a built-in method which contains the history of super class methods.
- Hence, we can use `super( )` to refer to super class constructor and methods from a sub class. So, `super( )` can be used as:

`super().init()` # call super class constructor

`super().init(arguments)` # call super class constructor and pass arguments.

`super().method()` # call super class method

Example: Write a python program to call the super class constructor in the sub class using `super( )`.

```
class Father:
    def __init__(self, p = 0):
        self.property = p
    def display(self):
        print("Father Property", self. property )
class Son(Father):
    def __init__(self,p1 = 0, p = 0):
        super(). init (p1)
        self.property1 = p
    def display(self):
        print("SonProperty",self.property+self.property1)
s=Son(200000,500000)
s.display()
```

Output:

Son Property 700000

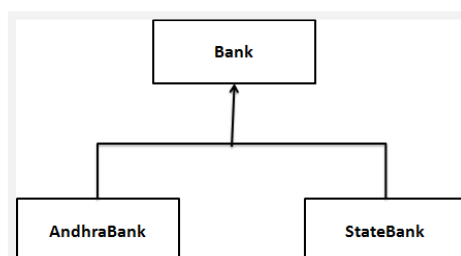
Types of Inheritance:

There are mainly 2 types of inheritance.

- a) Single inheritance
- b) Multiple inheritance

a) Single inheritance :

Deriving one or more sub classes from a single base class is called “**single inheritance**”. In single inheritance, we always have only one base class, but there can be n number of sub classes derived from it. For example, “Bank” is a single base class from where we derive “Andhra Bank” and “State Bank” as sub classes. This is called single inheritance.



Example:

```
class parent:
    def show(self):
        print("This is base class")
class child(parent):
    def display(self):
        print("This is derived class")
obj=child()
obj.show()
obj.display()
```

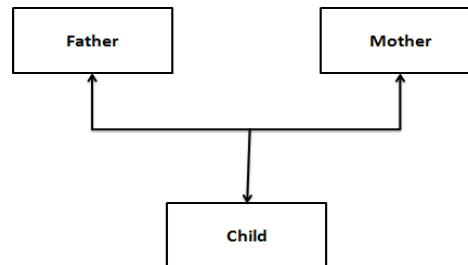
output:

```
This is base class
This is derived class
```

b) Multiple inheritance

Deriving sub classes from multiple (or more than one) base classes is called “**multiple inheritance**”. All the members of super classes are by default available to sub classes and the sub classes in turn can have their own members.

The best example for multiple inheritance is that parents are producing the children and the children inheriting the qualities of the parents.

**Example:**

```
class parent1:
    def add(self):
        print("addition")
class parent2:
    def sub(self):
        print("subtraction")
class child(parent1,parent2):
    def mul(self):
        print("multiplication")
obj=child()
obj.add()
obj.sub()
obj.mul()
```

output:

```
addition
subtraction
multiplication
```

Problem in Multiple inheritance:

- If the sub class has a constructor, it overrides the super class constructor and hence the super class constructor is not available to the sub class.
- But writing constructor is very common to initialize the instance variables.
- In multiple inheritance, let's assume that a sub class “C” is derived from two super classes “A” and “B” having their own constructors. Even the sub class “C” also has its constructor.

Example-1:

```
class A(object):
    def __init__(self):
        print("Class A")
    class B(object):
        def __init__(self):
            print("ClassB")
    class C(A,B,object):
        def __init__(self):
            super().__init__()
            print("Class C")
c1 = C()
```

Output:

```
Class A
Class C
```

Example-2:

```
class A(object):
    def __init__(self):
        super().__init__()
        print("Class A")
    class B(object):
        def __init__(self):
            super().__init__()
            print("ClassB")
    class C(A,B,object):
        def __init__(self):
            super().__init__()
            print("Class C")
c1 = C()
```

Output:

```
Class B
Class A
Class C
```

Method overloading :

Like other languages (for example, method overloading in C++) do, python does not support method overloading by default. But there are different ways to achieve method overloading in Python. The problem with method overloading in Python is that we may overload the methods but can only use the latest defined method.

Example :

```
#First product method.
#Takes two argument and print their product
def product(a, b):
    p = a * b
    print(p)
#Second product method
#Takes three argument and print their product
def product(a, b, c):
    p = a * b*c
    print(p)
#Uncommenting the below line shows an error
#product(4, 5)
#This line will call the second product method
product(4, 5, 5)
```

Output:

```
100
```

In the above code, we have defined two product method, but we can only use the second product method, as python does not support method overloading. We may define many methods of the same name and different arguments, but we can only use the latest defined method.

Calling the other method will produce an error. Like here calling `product(4, 5)` will produce an error as the latest defined product method takes three arguments.

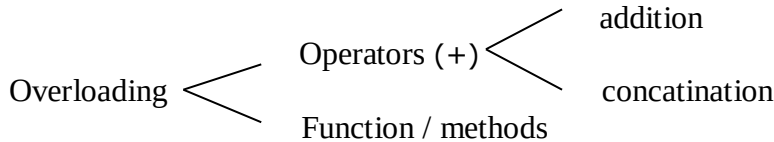
Thus, to overcome the above problem we can use different ways to achieve the method overloading.

Two methods cannot have the same name in Python. Method overloading in Python is a feature that allows the same operator to have different meanings there are different ways to achieve method overloading in Python.

They are of 2 types

1. Over Loading
2. Over Writing

1. Overloading:



Operators:

```

a=10
b=20
c=a+b
print(c)
x="CCN"
y="Branch"
z=x+" "+y
print(z)
  
```

Output:

```

30
CCN Branch
  
```

Function / methods :

Example :

```

class method:
    def sum(self,a=None,b=None,c=None):
        if(a!=None,b!=None,c!=None):
            s=a+b+c
            print(s)
        elif(a!=None and b!=None):
            s=a+b
            print(s)
        else:
            print("hello")
obj=method()
obj.sum(10,20,30)
  
```

output:

```

60
  
```

2. Over Writing:

It must be present one super class and sub class

Example 1 :

```

class branch:
    def student(self):
        print("CCN Branch")
class aditya(branch):
    def student(self):
        print("CSE Department")
obj=aditya()
obj.student()
  
```

output:

```

CSE Department
  
```

It contains same method name but perform different operations

Example 2:

```
class parent():
    def sum(self):
        print("parent class 1")
class parent2():
    def sum(self):
        print("parent class 2")
class child(parent,parent2):
    pass
klm=child()
klm.sum()
```

output:

parent class 1

Method Overriding:

Method overriding is a feature of object-oriented programming languages where a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. When a method in a subclass has the same name, same parameters or signature and same return type(or sub-type) as a method in its super-class, then the method in the subclass is said to override the method in the super-class.

The version of a method that is executed will be determined by the object that is used to invoke it. If an object of a parent class is used to invoke the method, then the version in the parent class will be executed, but if an object of the subclass is used to invoke the method, then the version in the child class will be executed. In other words, it is the type of the object being referred to (not the type of the reference variable) that determines which version of an overridden method will be executed.

Example:

```
class Parent():
    def __init__(self):
        self.value = "Inside Parent"
    def show(self):
        print(self.value)
class Child(Parent):
    def __init__(self):
        self.value = "Inside Child"
    def show(self):
        print(self.value)
obj=Child()
obj.show()
```

output:

Inside Child

Example:**program1:**

```
class branch:
    def student(self):
        print("ccn branch")
class aditya(branch):
    def student(self):
        print("CCN department")
obj=aditya()
obj.student()
```

output:

CCN department

```

program2:
class branch:
    def student(self):
        print("ccn branch")
class aditya(branch):
    def student(self):
        branch.student(self)
        print("CCN department")
obj=aditya()
obj.student()

```

output:
ccn branch
CCN department

Data hiding:

An object's attributes may or may not be visible outside the class definition. You need to name attributes with a double underscore prefix, and those attributes then are not be directly visible to outsiders.

It is a method used in object-oriented programming (OOP) to hide with the intention of hiding information/ data within a computer code. Data hiding helps computer programmers create classes with unique data sets and functions by avoiding unnecessary entrance from other classes in the program. Data hiding is also known as information When a data member is mentioned as private in the class, it is accessible only within the same class and inaccessible outside In Python, the data hiding isolates the features, data, class, program, etc from the users. The users do not get free access. This feature of data hiding enhances the security of the system and initiates better reliability. Only a few or very specific people get access that class. hiding This makes the attribute private/ inaccessible and hides them from users

To access private data member of class we use tricky syntax:-

objectName._className__attrName

for example (p1._Person__username)

Example 1:

```

class Person:
    #Hidden members of the class - Private member
    __username = 'programming'
    __password = 'python302'
    def Access(self):
        print("Username is:- {}".format(self.__username))
        print("password is:- {}".format(self.__password))
p1=Person()
p1.Access()
print("-----")
print(p1._Person__username)
print(p1._Person__password)

```

Output:

```

Username is:- programming
password is:- python302
-----
programming
python302

```

Example 2:

```

class counterclass:
    __privatecount=0
    def count(self):
        self.__privatecount+=1

```

```

        print(self.__privatecount)
    cl=counterclass()
    cl.count()
    cl.count()
    print(cl.count)

```

Output:

```

-
1
2
< bound method counterclass.count of < _main _ Counterclass object at
0x14bcd3762100>>

```

Example 3:

```

class Just Counter:
    __secretCount = 0
    def count(self):
        self. SecretCount += 1
        print(self. SecretCount)
counter = JustCounter()
counter.count()
counter.count()
print(counter.__Secret Count)

```

When the above code is executed, it produces the following result:

Python protects those members by internally changing the name to include the class name. You can access such attributes as ***object. _class Name attrName***. If you would replace your last line as following, then it works for you:

```

1
2
Trace back (most recent call last):
File "C:/Python27/JustCounter.py", line 9, in <module>
print(counter. Secret Count)
Attribute Error: Just Counter instance has no attribute 'secret Count'

```

Python protects those members by internally changing the name to include the class name. You can access such attributes as ***object. _class Name attrName***. If you would replace your last line as following, then it works for you:

```

.....
Print(counter. _Just Counter_secret Count)

```

When the above code is executed, it produces the following result:

```

1
2
2

```

File:

File is used to store data permanently and consol is used to store temporarily

Syntax for file creation:

```
fileobject=open(<filename>,<accessmode>,<bufferingmode>)
```

ex: f=open("test.txt", mode='r', encoding='utf-8')

File Creation:

File handling is an important part of any web application Python has several functions for
Aditya polytechnic Colleges ,surampalem.

creating, reading, updating, and deleting files There are four different methods (modes) for opening a file:

"r" - Read - Default value. Opens a file for reading, error if the file does not exist

"a" - Append - Opens a file for appending, creates the file if it does not exist

"w" - Write - Opens a file for writing, creates the file if it does not exist

"x" - Create - Creates the specified file, returns an error if the file exists

"t" - Text - Default value. Text mode

"b" - Binary - Binary mode (e.g. images)

"w+" - both r/w - Overwrites the existing file if the file exists If the file does not exist, creates a new file for reading and writing

"r+" - read and write - Opens a file for both reading and writing

"a+" - both r/w - Opens a file for both appending and reading

"wb" - writing only in binary format - Overwrites the file if the file exists If the file does not exist, creates a new file for writing

"ab" - append - Opens a file for appending in binary format

"rb" - read in binary - Opens a file for reading only in binary format

"rb+" - r/w in binary - Opens a file for both reading and writing in binary format

When we want to read from or write to a file, we need to open it first. When we are done, it needs to be closed so that the resources that are tied with the file are freed.

Hence, in Python, a file operation takes place in the following order:

- 1.Open a file
- 2.Read or write (perform operation)
- 3.Close the file

1.open a file:

Python has a built-in open() function to open a file. This function returns a file object, also called a handle, as it is used to read or modify the file accordingly.

```
>>> f = open("test.txt") # open file in current directory
>>> f = open("C:/Python38/README.txt") # specifying full path
```

2.close the file:

It is done using the close() method available in Python. Python has a garbage collector to clean up unreferenced objects but we must not rely on it to close the file.

```
>>> f = open("test.txt", encoding = 'utf-8') # perform file operations
>>> f.close()
```

This method is not entirely safe. If an exception occurs when we are performing some operation with the file, the code exits without closing the file.

A safer way is to use a try...finally block.

```
>>> try:
    f = open("test.txt", encoding = 'utf-8') #perform file operations
    finally:
        f.close()
```

This way, we are guaranteeing that the file is properly closed even if an exception is raised that causes program flow to stop.

The best way to close a file is by using the with statement. This ensures that the file is closed when the block inside the with statement is exited.

We don't need to explicitly call the close() method. It is done internally.

```
>>>with open("test.txt", encoding = 'utf-8') as f: # perform file operations
```

Example1:

```
f=open("CCN.txt","w") #you must add a parameter to the open() function
f.write("CCN students")
if f:
    print("created")
f.close()
output: created
```

Example 2:

```
f = open("demofile2.txt", "a")
f.write("Now the file has more content!")
f.close()
f = open("demofile2.txt", "r")    #open and read the file after the appending
print(f.read())
f = open("demofile2.txt", "a")
f.write("Now the file has more content!")
f.close()
f= open("demofile2.txt", "r")    #open and read the file after the appending
print(f.read())
```

output:

Now the file has more content!Now the file has more content!Now the file has more content!Now the file has more content!Now the file has more content!Now the file has more content!

Errors and Exceptions:

As human beings, we commit several errors. A software developer is also a human being and hence prone to commit errors wither in the design of the software or in writing the code. The errors in the software are called “**bugs**” and the process of removing them are called “**debugging**”. In general, we can classify errors in a program into one of these three types:

- a) Compile-time errors
- b) Runtime errors
- c) Logical errors

a) **Compile-time errors**

These are **syntactical errors** found in the code, due to which a program fails to compile. For example, forgetting a colon in the statements like if, while, for, def, etc. will result in compile-time error. Such errors are detected by python compiler and the line number along with error description is displayed by the python compiler.

Example: A Python program to understand the compile-time error.

```
a = 1
if a == 1
Print("hello")
```

Output:

```
File ex.py, line 3
If a == 1
    ^
Syntax Error: invalid syntax
```

b) Runtime errors

When PVM cannot execute the byte code, it flags runtime error. For example, insufficient memory to store something or inability of PVM to execute some statement come under runtime errors. Runtime errors are not detected by the python compiler. They are detected by the PVM, Only at runtime.

Example: A Python program to understand the compile-time error.

```
print("hai"+25)
```

Output:

```
Trace back (most recent call last):
  File "<pyshell#0>", line 1, in <module> print "hai"+25
TypeError: cannot concatenate 'str' and 'int' objects
```

c) Logical errors

These errors depict flaws in the logic of the program. The programmer might be using a wrong formula or the design of the program itself is wrong. Logical errors are not detected either by Python compiler or PVM. The programmer is solely responsible for them. In the following program, the programmer wants to calculate incremented salary of an employee, but he gets wrong output, since he uses wrong formula.

Example: A Python program to increment the salary of an employee by 15%.

```
def increment(sal):
    sal = sal * 15/100
    return sal
sal = increment(5000)
print("Salary after Increment is", sal)
```

Output:

```
Salary after Increment is 750
```

From the above program the formula for salary is wrong, because only the increment but it is not adding it to the original salary. So, the correct formula would be:

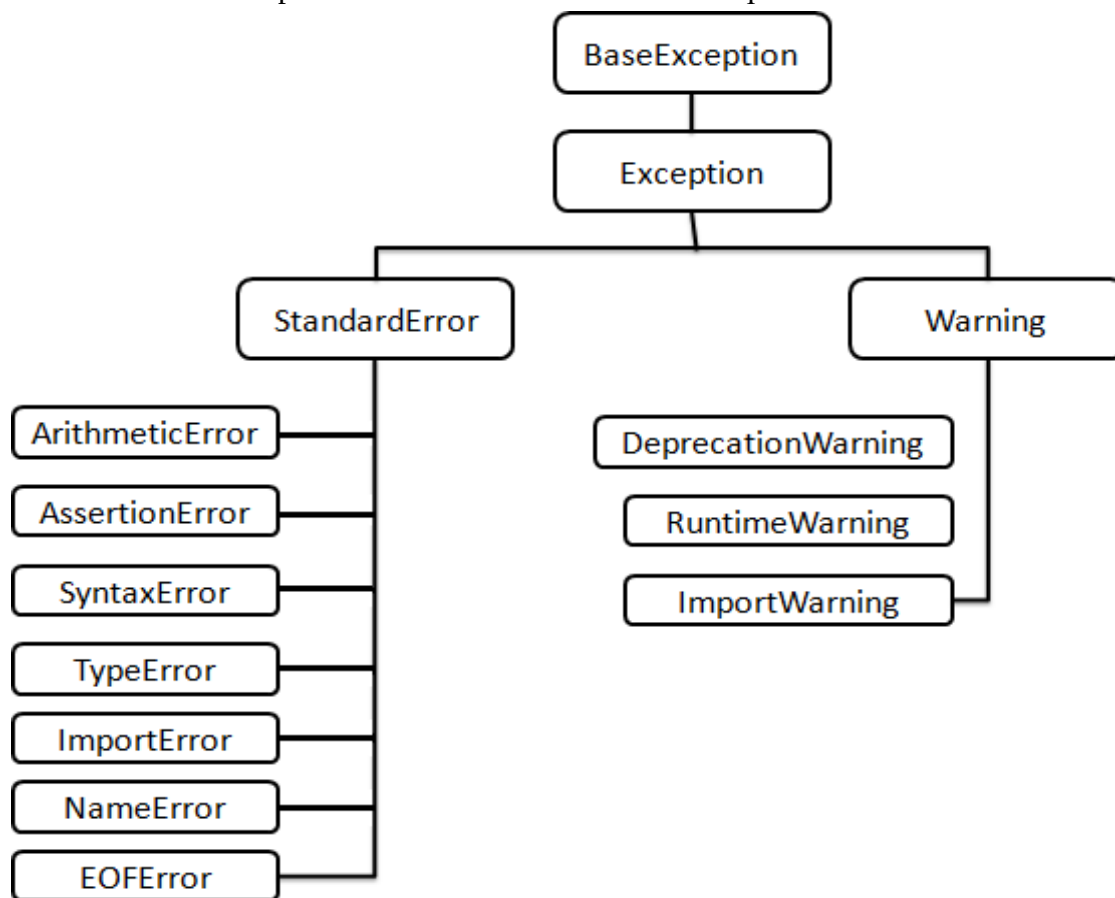
```
sal = sal + sal * 15/100
```

- ✓ Compile time errors and runtime errors can be eliminated by the programmer by modifying the program source code.
- ✓ In case of runtime errors, when the programmer knows which type of error occurs, he has to handle them using **exception handling mechanism**.

Exceptions:

- An exception is a runtime error which can be handled by the programmer.
- That means if the programmer can guess an error in the program and he can do something to eliminate the harm caused by that error, then it is called an **“exception”**.
- If the programmer cannot do anything in case of an error, then it is called an **“error”** and not an exception.
- All exceptions are represented as classes in python. The exceptions which are already available in python are called **“built-in”** exceptions. The base class for all built-in exceptions is **“Base Exception”** class.
- From Base Exception class, the sub class **“Exception”** is derived. From Exception class, the sub classes **“Standard Error”** and **“Warning”** are derived.
- All errors (or exceptions) are defined as sub classes of Standard Error. An error should be compulsory handled otherwise the program will not execute.
- Similarly, all warnings are derived as sub classes from **“Warning”** class. A warning represents a caution and even though it is not handled, the program will execute. So, warnings can be neglected but errors cannot neglect.

- Just like the exceptions which are already available in python language, a programmer can also create his own exceptions, called “user-defined” exceptions.
- When the programmer wants to create his own exception class, he should derive his class from Exception class and not from “Base Exception” class.



Exception Handling:

- The purpose of handling errors is to make the program *robust*. The word “robust” means “strong”. A robust program does not terminate in the middle.
- Also, when there is an error in the program, it will display an appropriate message to the user and continue execution.
- Designing such programs is needed in any software development.
- For that purpose, the programmer should handle the errors. When the errors can be handled, they are called exceptions.

To handle exceptions, the programmer should perform the following four steps:

Step 1: The programmer should observe the statements in his program where there may be a possibility of exceptions. Such statements should be written inside a “try” block. A try block looks like as follows:

```

try:
    statements
  
```

The greatness of try block is that even if some exception arises inside it, the program will not be terminated. When PVM understands that there is an exception, it jumps into an “except” block.

Step 2: The programmer should write the “except” block where he should display the

exception details to the user. This helps the user to understand that there is some error in the program. The programmer should also display a message regarding what can be done to avoid this error. Except block looks like as follows:

except exception name:
statements

The statements written inside an except block are called “handlers” since they handle the situation when the exception occurs.

Step 3: If no exception is raised, the statements inside the “else” block is executed. Else block looks like as follows:

else:
statements

Step 4: Lastly, the programmer should perform clean up actions like closing the files and terminating any other processes which are running. The programmer should write this code in the finally block. Finally block looks like as follows:

finally:
statements

The specialty of finally block is that the statements inside the finally block are executed irrespective of whether there is an exception or not. This ensures that all the opened files are properly closed and all the running processes are properly terminated. So, the data in the files will not be corrupted and the user is at the safe-side.

Here, the complete exception handling syntax will be in the following format:

```
try:
    statements
except Exception1:
    statements
except Exception2:
    statements
else:
    statements
finally:
    statements
```

The following points are followed in exception handling:

- ✓ A single try block can be followed by several except blocks.
- ✓ Multiple except blocks can be used to handle multiple exceptions.
- ✓ We cannot write except blocks without a try block.
- ✓ We can write a try block without any except blocks.
- ✓ Else block and finally blocks are not compulsory.
- ✓ When there is no exception, else block is executed after try block.
- ✓ Finally block is always executed.

Example: A python program to handle IO Error produced by open() function.

```
import sys
try:
    f = open('myfile.txt','r')
    s = f.readline()
    print(s)
    f.close()
except IOError as e:
    print((IO error",e.strerror)
except:
    print("Unexpected error:")
```

Output:

I/O error No such file or directory

In the if the file is not found, then IO Error is raised. Then 'except' block will display a message: 'I/O error'. if the file is found, then all the lines of the file are read using read line () method.

List of Standard Exceptions

Exception Name	Description
Exception	Base class for all exceptions
Stop Iteration	Raised when the next() method of an iterator does not point to any object.
System Exit	Raised by the sys.exit() function.
Standard Error	Base class for all built-in exceptions except Stop Iteration and System Exit.
Arithmetic Error	Base class for all errors that occur for numeric calculation.
Overflow Error	Raised when a calculation exceeds maximum limit for a numeric type.
Floating Point Error	Raised when a floating point calculation fails.
Zero Division Error	Raised when division or modulo by zero takes place for all numeric types.
Assertion Error	Raised in case of failure of the Assert statement.
Attribute Error	Raised in case of failure of attribute reference or assignment.
EOF Error	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
Import Error	Raised when an import statement fails.
Keyboard Interrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.
Lookup Error	Base class for all lookup errors.
Index Error	Raised when an index is not found in a sequence.
Key Error	Raised when the specified key is not found in the dictionary.
Name Error	Raised when an identifier is not found in the local or global namespace.
Unbound Local Error	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
Environment	Base class for all exceptions that occur outside the Python environment.

nt Error	
IO Error	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
OS Error	Raised for operating system-related errors.
Syntax Error	Raised when there is an error in Python syntax.
Indentation Error	Raised when indentation is not specified properly.
System Error	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
System Exit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
Type Error	Raised when an operation or function is attempted that is invalid for the specified data type.
Value Error	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
Runtime Error	Raised when a generated error does not fall into any category.
Not Implemented Error	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

The Except Block:

The “except” block is useful to catch an exception that is raised in the try block. When there is an exception in the try block, then only the except block is executed. It is written in various formats.

1. To catch the exception which is raised in the try block, we can write except block with the Exception class name as:

except Exception class:

2. We can catch the exception as an object that contains some description about the exception.

except Exception class as obj:

3. To catch multiple exceptions, we can write multiple catch blocks. The other way is to use a single except block and write all the exceptions as a tuple inside parentheses as:

except (Exceptionclass1, Exceptionclass2, ...):

4. To catch any type of exception where we are not bothered about which type of exception it is, we can write except block without mentioning any Exception class name as:

except:

Example:

```
try:
    f = open('myfile.txt','w')
    a=input("Enter a value ")
    b=input("Enter a value ")
```

```

c=a/float(b)
s = f.write(str(c))
print("Result is stored")
except ZeroDivisionError:
    print("Division is notpossible")except:
    print("Unexpectederror:")
finally:
    f.close()

```

Output:

```

Enter a value 1
Enter a value 5
Result is stored

```

Raising an Exception

You can raise exceptions in several ways by using the raise statement. The general syntax for the **raise** statement is as follows.

```
raise [Exception [, args [, traceback]]]
```

Here, *Exception* is the type of exception (For example, `NameError`) and *argument* is a value for the exception argument. The argument is optional; if not supplied, the exception argument is `None`.

For Example, If you need to determine whether an exception was raised but don't intend to handle it, a simpler form of the raise statement allows you to re-raise the exception:

```

try:
    raise NameError('HiThere')
except Name Error:
    Print('An exception flewby!')raise

```

User-Defined Exceptions:

- Like the built-in exceptions of python, the programmer can also create his own exceptions which are called “**User-defined exceptions**” or “**Custom exceptions**”. We know Python offers many exceptions which will raise in different contexts.
- But, there may be some situations where none of the exceptions in Python are useful for the programmer. In that case, the programmer has to create his/her own exception and raise it.
- For example, let's take a bank where customers have accounts. Each account is characterized should by customer name and balance amount.
- The rule of the bank is that every customer should keep minimum Rs. 2000.00 as balance amount in his account.
- The programmer now is given a task to check the accounts to know every customer is maintaining minimum balance of Rs. 2000.00 or not.
- If the balance amount is below Rs. 2000.00, then the programmer wants to raise an exception saying „Balance amount is less in the account of so and so person.“ This will be helpful to the bank authorities to find out the customer.
- So, the programmer wants an exception that is raised when the balance amount in an account is less than Rs. 2000.00. Since there is no such exception available in python, the programmer has to create his/her own exception.
- For this purpose, he/she has to follow these steps:

1. Since all exceptions are classes, the programmer is supposed to create his own exception as a class. Also, he should make his class as a sub class to the in-built “Exception” class.

```
class My Exception(Exception):
```

```
def __init__(self, arg):  
    self.msg = arg
```

Here, My Exception class is the sub class for “Exception” class. This class has a constructor where a variable “msg” is defined. This “msg” receives a message passed from outside through “arg”.

2. The programmer can write his code; maybe it represents a group of statements or a function. When the programmer suspects the possibility of exception, he should raise his own exception using “raise” statement as:

```
raise MyException('message')
```

Here, raise statement is raising My Exception class object that contains the given “message”.

3. The programmer can insert the code inside a “try” block and catch the exception using “except” block as:

```
try:  
    code  
except My Exception as me:  
    print me
```

Here, the object “me” contains the message given in the raise statement. All these steps are shown in below program.

Example 1:

```
class MyException(Exception):  
    def __init__(self, arg):  
        self.msg = arg  
    def check(dict):  
        for k,v in dict.items():  
            print("Name=",k,"Balance=",)  
            if v<2000.00:  
                raise MyException("Balance amount is less in the account of "+k)  
  
bank={"ravi":5000.00,"ramu":8500.00,"raju":1990.00}  
try:  
    check(bank)  
except My Exception as me:  
    print(me.msg)
```

output:

```
Name= ramu Balance= 8500.0  
Name= ravi Balance= 5000.0  
Name= raju Balance= 1990.0  
Balance amount is less in the account of raju
```

Example 2:

```
a=int(input("enter your age"))  
try:  
    if(a<18):  
        raise ValueError  
    print("Eligible for vote")  
except ValueError:  
    print("you are not eligible for vote")
```

output:

enter your age 16
you are not eligible for vote

Example 3:

```
#define Python user-defined exceptions
class Error(Exception):
    pass
class zerodivision(Error):
    pass
try:
    num=int(input("Enter a number: "))
    if num==0:
        raise zerodivision
    print("you entered {} which is other than 0".format(num))
except zerodivision:
    print("Input value is zero, try again!")
```

output:

Enter a number: 0
Input value is zero, try again!

